

CODE

```
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <SPI.h>

#define TFT_CS 10
#define TFT_DC 9
#define TFT_RST 8

Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC, TFT_RST);

#define ECG_PIN A0
#define LO_PLUS 2
#define LO_MINUS 3

#define SCREEN_W 320
#define SCREEN_H 240

#define PLOT_X 0
#define PLOT_Y 30
#define PLOT_W 320
#define PLOT_H 180
#define BASELINE (PLOT_Y + PLOT_H / 2)

#define GRID_COLOR 0x1800
#define GRID_SPACING 20

#define COLOR_BG ILI9341_BLACK
#define COLOR_WAVE 0x07E0
#define COLOR_HEADER 0x1082
#define COLOR_TEXT ILI9341_WHITE
#define COLOR_ALERT ILI9341_RED

#define R_THRESHOLD 420
#define MIN_RR_MS 300

int xPos = PLOT_X;
int prevY = BASELINE;
int bpm = 0;
bool leadOff = false;

unsigned long lastPeakTime = 0;
unsigned long lastBpmUpdate = 0;
int peakCount = 0;
unsigned long peakTimes[8];
int peakIndex = 0;
bool prevAbove = false;

int getBpm() {
  if (peakCount < 2) return 0;
```

```

int useCount = min(peakCount - 1, 4);
unsigned long totalRR = 0;

for (int j = 1; j <= useCount; j++) {
    int idx1 = (peakIndex - j + 8) % 8;
    int idx2 = (peakIndex - j - 1 + 8) % 8;
    totalRR += peakTimes[idx1] - peakTimes[idx2];
}

int calcBpm = (int)(60000.0 / (totalRR / (float)useCount));
return constrain(calcBpm, 30, 220);
}

void drawGrid() {
    tft.fillRect(PLOT_X, PLOT_Y, PLOT_W, PLOT_H, COLOR_BG);

    for (int x = PLOT_X; x < PLOT_X + PLOT_W; x += GRID_SPACING) {
        tft.drawFastVLine(x, PLOT_Y, PLOT_H, GRID_COLOR);
    }
    for (int y = PLOT_Y; y < PLOT_Y + PLOT_H; y += GRID_SPACING) {
        tft.drawFastHLine(PLOT_X, y, PLOT_W, GRID_COLOR);
    }
    tft.drawFastHLine(PLOT_X, BASELINE, PLOT_W, 0x3186);
}

void drawHeader() {
    tft.fillRect(0, 0, SCREEN_W, PLOT_Y, COLOR_HEADER);

    tft.setTextColor(COLOR_WAVE);
    tft.setTextSize(2);
    tft.setCursor(4, 7);
    tft.print("ECG");

    updateBpmDisplay();
}

void updateBpmDisplay() {
    tft.fillRect(160, 0, 160, PLOT_Y, COLOR_HEADER);

    tft.setTextSize(2);
    if (!leadOff) {
        tft.setTextColor(COLOR_ALERT);
        tft.setCursor(168, 7);
        tft.print("OFF");
    } else {
        tft.setTextColor(COLOR_TEXT);
        tft.setCursor(100, 7);
        tft.print("BPM: ");
        bpm = getBpm();
        if (bpm > 0) {

```

```

    tft.setTextColor(COLOR_WAVE);
    tft.print(bpm);
} else {
    tft.setTextColor(0x7BEF);
    tft.print("----");
}
}
}

void eraseAhead(int x) {
    int eraseX = (x + 4) % PLOT_W;
    tft.fillRect(PLOT_X + eraseX, PLOT_Y, 4, PLOT_H, COLOR_BG);
    for (int gx = PLOT_X + eraseX; gx < PLOT_X + eraseX + 4; gx++) {
        if ((gx % GRID_SPACING) == 0)
            tft.drawFastVLine(gx, PLOT_Y, PLOT_H, GRID_COLOR);
    }
    for (int gy = PLOT_Y; gy < PLOT_Y + PLOT_H; gy += GRID_SPACING) {
        tft.drawPixel(PLOT_X + eraseX, gy, GRID_COLOR);
    }
}

int adcToY(int adcVal) {
    int y = map(adcVal, 0, 1023, PLOT_Y + PLOT_H - 5, PLOT_Y + 5);
    return constrain(y, PLOT_Y + 2, PLOT_Y + PLOT_H - 2);
}

void detectPeak(int adcVal) {
    bool above = (adcVal > R_THRESHOLD);

    if (above && !prevAbove) {
        unsigned long now = millis();
        if (now - lastPeakTime > MIN_RR_MS) {
            peakTimes[peakIndex % 8] = now;
            peakIndex++;
            peakCount = peakIndex;
            lastPeakTime = now;
        }
    }
    prevAbove = above;
}

void setup() {
    Serial.begin(115200);

    pinMode(LO_PLUS, INPUT);
    pinMode(LO_MINUS, INPUT);

    tft.begin();
    tft.setRotation(2);
    tft.fillScreen(COLOR_BG);

```

```

drawGrid();
drawHeader();

delay(500);
}

void loop() {
  bool newLeadOff = (digitalRead(LO_PLUS) == HIGH || digitalRead(LO_MINUS) == HIGH);

  if (newLeadOff != leadOff) {
    leadOff = newLeadOff;
    updateBpmDisplay();
  }

  if (leadOff) {
    int y = BASELINE;
    tft.drawPixel(PLOT_X + xPos, y, COLOR_ALERT);
    Serial.println(0);
  } else {
    int adcVal = analogRead(ECG_PIN);

    Serial.println(adcVal);

    detectPeak(adcVal);

    int curY = adcToY(adcVal);

    eraseAhead(xPos);

    tft.drawLine(
      PLOT_X + ((xPos == 0 ? PLOT_W - 1 : xPos - 1)), prevY,
      PLOT_X + xPos, curY,
      COLOR_WAVE
    );

    prevY = curY;
  }

  xPos++;
  if (xPos >= PLOT_W) {
    xPos = 0;
    prevY = BASELINE;
  }

  updateBpmDisplay();

  delay(4);
}

```

